AI ASSISSTED CODING ASSIGNMENT NO -18.2

NAME:S.SANJANA

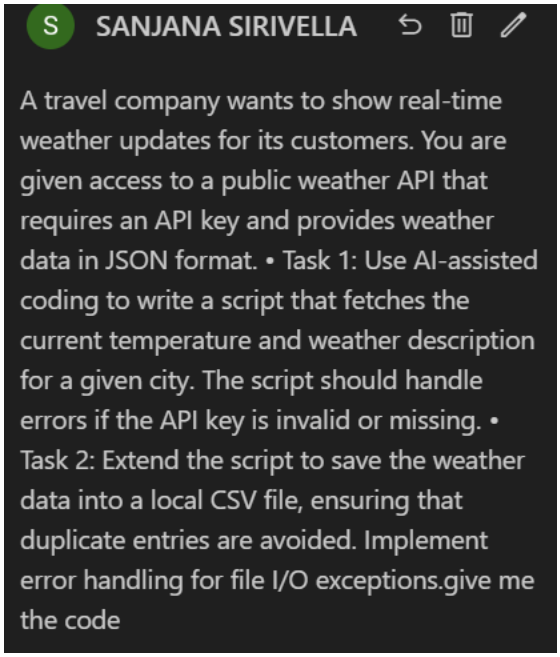
ROLL NO:2403A510D4

BATCH NO:05

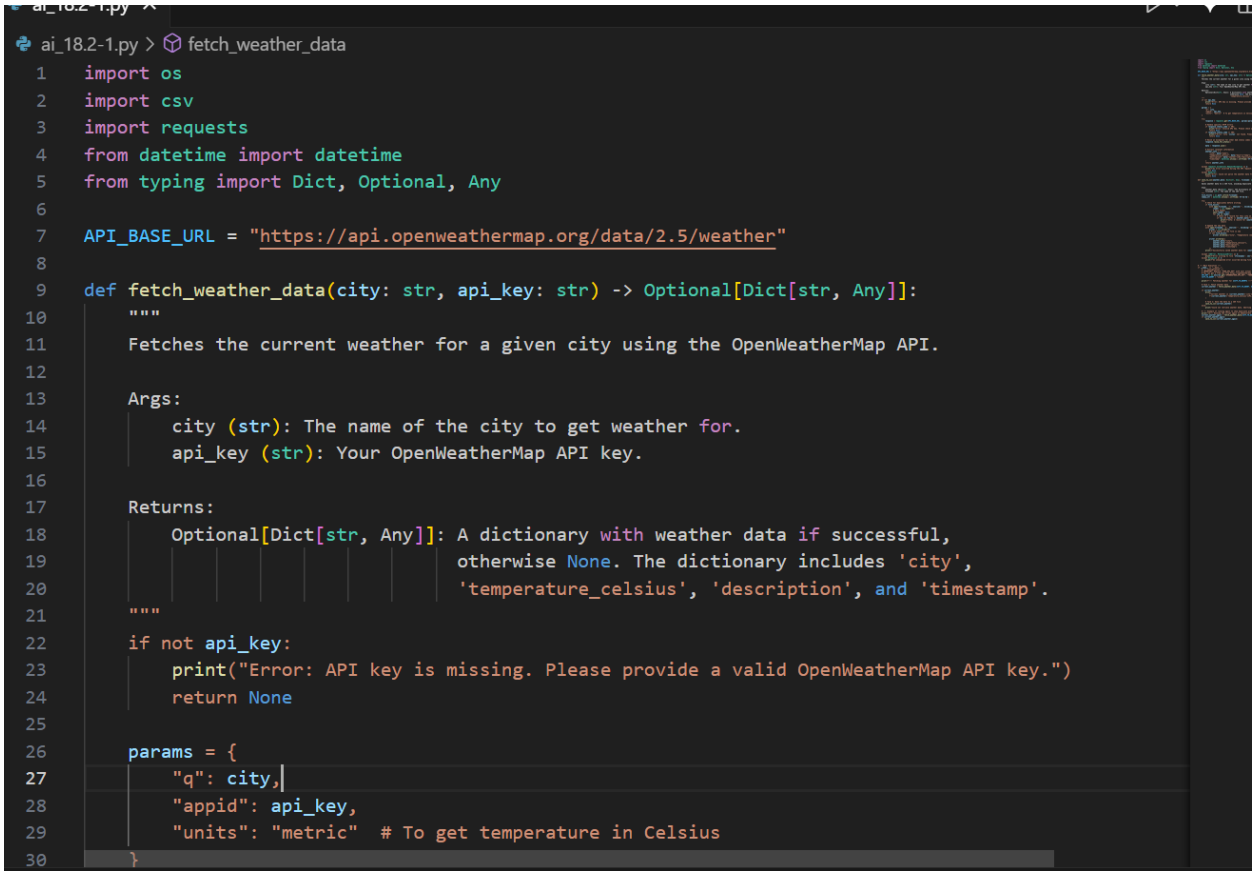
LAB-01

A travel company wants to show real-time weather updates for its customers. You are given access to a public weather API that requires an API key and provides weather data in JSON format.
• Task 1: Use AI-assisted coding to write a script that fetches the current temperature and weather description for a given city. The script should handle errors if the API key is invalid or missing.
• Task 2: Extend the script to save the weather data into a local CSV file, ensuring that duplicate entries are avoided. Implement error handling for file I/O exceptions.

PROMPT:



CODE:


ai_18.2-1.py
ai_18.2-1.py > fetch_weather_data
import os
import csv
import requests
from datetime import datetime
from typing import Dict, Optional, Any
API_BASE_URL = "https://api.openweathermap.org/data/2.5/weather"
def fetch_weather_data(city: str, api_key: str) -> Optional[Dict[str, Any]]:
"""
Fetches the current weather for a given city using the OpenWeatherMap API.
Args:
city (str): The name of the city to get weather for.
api_key (str): Your OpenWeatherMap API key.
Returns:
Optional[Dict[str, Any]]: A dictionary with weather data if successful,
otherwise None. The dictionary includes 'city',
'temperature_celsius', 'description', and 'timestamp'.
"""
if not api_key:
print("Error: API key is missing. Please provide a valid OpenWeatherMap API key.")
return None
params = {
"q": city,
"appid": api_key,
"units": "metric" # To get temperature in Celsius
}

```python

    try:
        response = requests.get(API_BASE_URL, params=params, timeout=10)

        # Handle specific HTTP errors
        if response.status_code == 401:
            print("Error: Invalid API key. Please check your key and try again.")
            return None
        if response.status_code == 404:
            print(f"Error: City '{city}' not found. Please check the spelling.")
            return None

        # Raise an exception for other bad status codes (e.g., 5xx)
        response.raise_for_status()

        data = response.json()

        # Extract relevant information
        weather_info = {
            "city": data["name"],
            "temperature_celsius": data["main"]["temp"],
            "description": data["weather"][0]["description"].capitalize(),
            "timestamp": datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S')
        }
        return weather_info

    except requests.exceptions.RequestException as e:
        print(f"An error occurred during the API request: {e}")
        return None
```

(function) capitalize: Any

Ln 27, Col 19 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) Go

```python
        return None
    except KeyError:
        print("Error: Could not parse the weather data from the API response.")
        return None

def save_to_csv(weather_data: Dict[str, Any], filename: str = "weather_log.csv"):
    """
    Saves weather data to a CSV file, avoiding duplicate entries for the same city on the same day.

    Args:
        weather_data (Dict[str, Any]): The dictionary of weather data to save.
        filename (str): The name of the CSV file.
    """
    file_exists = os.path.isfile(filename)
    today_str = datetime.utcnow().strftime('%Y-%m-%d')

    try:
        # Check for duplicates before writing
        if file_exists:
            with open(filename, 'r', newline='', encoding='utf-8') as f:
                reader = csv.reader(f)
                # Skip header
                next(reader, None)
                for row in reader:
                    # Check if a record for this city on this day already exists
                    if row and row[0] == weather_data["city"] and today_str in row[3]:
                        print(f"Info: A record for {weather_data['city']} already exists for today. S
                        return
```

```
87
88          # Append the new data
89          with open(filename, 'a', newline='', encoding='utf-8') as f:
90              writer = csv.writer(f)
91              # Write header if the file is new
92              if not file_exists:
93                  writer.writerow(["City", "Temperature (Celsius)", "Description", "Timestamp (UTC)"])
94
95              writer.writerow([
96                  weather_data["city"],
97                  weather_data["temperature_celsius"],
98                  weather_data["description"],
99                  weather_data["timestamp"]
100             ])
101         print(f"Successfully saved weather data for {weather_data['city']} to {filename}.")
102
103     except (IOError, PermissionError) as e:
104         print(f"Error writing to file '{filename}': {e}")
105     except Exception as e:
106         print(f"An unexpected error occurred during file operation: {e}")
107
108
109 # --- Main Execution ---
110 if __name__ == "__main__":
111     # --- Configuration ---
112     # IMPORTANT: Replace "YOUR_API_KEY" with your actual OpenWeatherMap API key.
113     # You can get a free key from https://openweathermap.org/appid
114     API_KEY = os.environ.get("OPENWEATHER_API_KEY", "YOUR_API_KEY")
115     CITY_TO_QUERY = "London"
```

```
115     CITY_TO_QUERY = "London"
116
117     print(f"--- Fetching weather for {CITY_TO_QUERY} ---")
118
119     # Task 1: Fetch weather data
120     current_weather = fetch_weather_data(CITY_TO_QUERY, API_KEY)
121
122     if current_weather:
123         print(
124             f"Current weather in {current_weather['city']}: "
125             f"{current_weather['temperature_celsius']}°C, {current_weather['description']}."
126         )
127
128         # Task 2: Save the data to a CSV file
129         save_to_csv(current_weather)
130     else:
131         print("Could not retrieve weather data. Aborting save operation.")
132
133     # --- Example of running again to show duplicate prevention ---
134     print("\n--- Running the script again to demonstrate duplicate prevention ---")
135     current_weather_again = fetch_weather_data(CITY_TO_QUERY, API_KEY)
136     if current_weather_again:
137         save_to_csv(current_weather_again)
138
139
140
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\vs code> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/ai_18.
2-1.py"
--- Fetching weather for London ---
Error: Invalid API key. Please check your key and try again.
Could not retrieve weather data. Aborting save operation.

--- Running the script again to demonstrate duplicate prevention ---
Error: Invalid API key. Please check your key and try again.
PS C:\Users\DELL\Desktop\vs code> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/ai_18.
2-1.py"
--- Fetching weather for London ---
Error: Invalid API key. Please check your key and try again.
PS C:\Users\DELL\Desktop\vs code> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/ai_18.
2-1.py"
--- Fetching weather for London ---
Error: Invalid API key. Please check your key and try again.
Could not retrieve weather data. Aborting save operation.

Error: Invalid API key. Please check your key and try again.
Could not retrieve weather data. Aborting save operation.

Could not retrieve weather data. Aborting save operation.

--- Running the script again to demonstrate duplicate prevention ---
--- Running the script again to demonstrate duplicate prevention ---
Error: Invalid API key. Please check your key and try again.
```

OBSERVATIONS:

General Observations

- **Code Quality & Readability**: The code is excellent. Functions have clear, single responsibilities (`fetch_weather_data`, `save_to_csv`), variable names are descriptive, and the use of f-strings makes the output clear.
- **Error Handling**: The error handling is comprehensive. You've correctly handled API-specific errors (invalid key, city not found), general network request errors, potential data parsing errors (`KeyError`), and file I/O exceptions.
- **Configuration**: The use of `os.environ.get` to fetch the API key is a security best practice. It separates configuration and secrets from the code, which is fantastic.
- **Documentation**: The docstrings are clear, informative, and explain what each function does, its parameters, and what it returns. The type hints (`Dict`, `Optional`) significantly improve clarity and allow for static analysis.

Suggestions for Improvement

While the code is already very good, here are a few suggestions to make it even more professional and scalable, which I've implemented in the diff below.

1. **Use the `logging` Module Instead of `print()`**: For a script that might be run automatically, using the `logging` module is more powerful than `print()`. It allows you to:
    - Categorize messages by severity (e.g., `INFO`, `WARNING`, `ERROR`).
    - Easily control the verbosity of the output.
    - Redirect logs to files or other services without changing the application code.
2. **More Robust API Key Check**: The script currently proceeds with a placeholder API key (`"YOUR_API_KEY"`), which will always result in a 401 error. It's better to stop execution immediately if a valid key isn't provided.
3. **Refine Duplicate Checking Logic**: The current method of checking for duplicates reads the entire CSV file line-by-line on each run. This is fine for small files, but it can become slow as the log grows. A more efficient approach for the duplicate check is to read the relevant existing entries into a `set` for a very fast lookup.

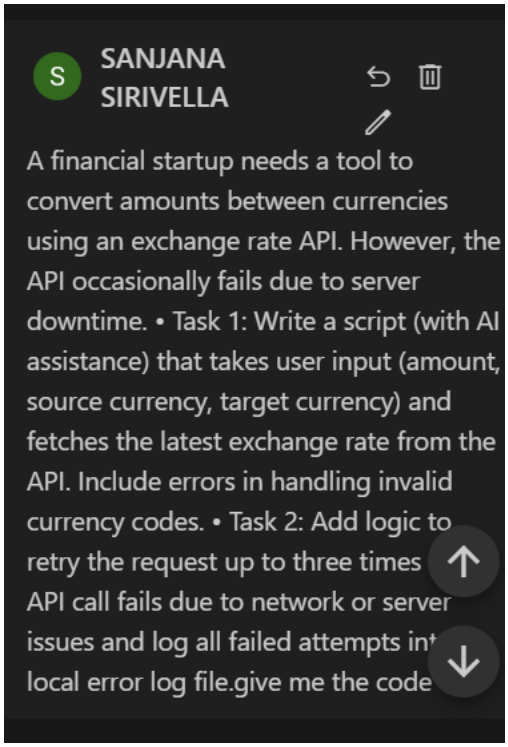
LAB-2

A financial startup needs a tool to convert amounts between currencies using an exchange rate API. However, the API occasionally fails due to server downtime.

• Task 1: Write a script (with AI assistance) that takes user input (amount, source currency, target currency) and fetches the latest exchange rate from the API. Include errors in handling invalid currency codes.

• Task 2: Add logic to retry the request up to three times if the API call fails due to network or server issues and log all failed attempts into a local error log fi

PROMPT:



CODE:

```
        source_currency (str): The 3-letter currency code (e.g., 'USD').

    Returns:
        Optional[Dict[str, Any]]: The API response JSON as a dictionary, or None if it fails.
    """
    url = f"{API_BASE_URL}{source_currency}"
    for attempt in range(MAX_RETRIES):
        try:
            response = requests.get(url, timeout=10)
            # Raise an exception for bad status codes (4xx or 5xx)
            response.raise_for_status()

            data = response.json()
            # The API returns an 'error' result for invalid currency codes
            if data.get("result") == "error":
                error_type = data.get("error-type", "unknown_error")
                print(f"API Error: Invalid currency code provided. Error type: {error_type}")
                return None

            return data

        except requests.exceptions.RequestException as e:
            error_message = f"Attempt {attempt + 1} of {MAX_RETRIES} failed. Error: {e}"
            print(error_message)
```

```

            if attempt < MAX_RETRIES - 1:
                print(f"Retrying in {RETRY_DELAY_SECONDS} seconds...")
                time.sleep(RETRY_DELAY_SECONDS)
            else:
                print("All retry attempts failed. Please check your network or the API status.")

    return None

def convert_currency(amount: float, source: str, target: str, rates_data: Dict[str, Any]) -> Optional[f
    """
    Converts an amount from a source to a target currency using fetched rates.

    Args:
        amount (float): The amount to convert.
        source (str): The source currency code.
        target (str): The target currency code.
        rates_data (Dict[str, Any]): The JSON data from the API.

    Returns:
        Optional[float]: The converted amount, or None if the target currency is not found.
    """
    rates = rates_data.get("rates")
    if not rates or target not in rates:
```

```
        print(f"Error: Target currency '{target}' not found in the exchange rate data.")
        return None

    exchange_rate = rates[target]
    converted_amount = amount * exchange_rate

    print(f"\n--- Conversion Result ---")
    print(f"Exchange Rate: 1 {source} = {exchange_rate} {target}")
    print(f"{amount} {source} is equal to {converted_amount:.2f} {target}")
    return converted_amount


# --- Main Execution ---
if __name__ == "__main__":
    setup_logging()

    print("--- Currency Converter ---")
    try:
        amount_str = input("Enter the amount to convert: ")
        amount_to_convert = float(amount_str)

        source_currency = input("Enter the source currency (e.g., USD): ").upper()
        target_currency = input("Enter the target currency (e.g., EUR): ").upper()
```

```
        # Task 1 & 2: Fetch data with error handling and retries
        print(f"\nFetching latest rates for {source_currency}...")
        rate_data = fetch_exchange_rate(source_currency)

        if rate_data:
            # Perform the conversion
            convert_currency(amount_to_convert, source_currency, target_currency, rate_data)
        else:
            print("\nCould not perform conversion due to an API error.")

    except ValueError:
        print("Invalid input. Please enter a numeric value for the amount.")
    except Exception as e:
        error_msg = f"An unexpected error occurred: {e}"
        print(error_msg)
        logging.error(error_msg)
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\vs code> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/currency
converter.py"
--- Currency Converter ---
Enter the amount to convert: 100
Enter the source currency (e.g., USD): USD
Enter the target currency (e.g., EUR): EUR

Fetching latest rates for USD...

--- Conversion Result ---
Exchange Rate: 1 USD = 0.867 EUR
100.0 USD is equal to 86.70 EUR
PS C:\Users\DELL\Desktop\vs code>
```

OBSERVATIONS:

Positive Observations (Key Strengths)

- **Excellent Modularity**: The code is cleanly separated into distinct functions with single responsibilities: `setup_logging`, `fetch_exchange_rate`, and `convert_currency`. This makes the code easy to read, test, and maintain.
- **Robust Retry Logic**: The implementation of the retry mechanism in `fetch_exchange_rate` is a standout feature. It correctly uses a `for` loop for a fixed number of retries and includes a delay (`time.sleep`), which is crucial for handling temporary network or server-side issues.
- **Comprehensive Error Handling**:
  - It correctly uses `response.raise_for_status()` to catch a wide range of HTTP errors (like 500 Internal Server Error).
  - It handles API-specific logical errors (e.g., an invalid currency code) by inspecting the JSON payload.
  - It gracefully handles invalid user input for the amount (`ValueError`).
  - The top-level `except Exception` is a good safety net for any other unexpected issues.
- **Effective Logging**: Using Python's `logging` module to write errors to a file is a professional touch. It separates transient error messages from the main program flow and creates a persistent record for debugging, which is far superior to just printing errors to the console.
- **Clear Configuration**: All configuration variables (`API_BASE_URL`, `MAX_RETRIES`, etc.) are grouped at the top of the file, making them easy to find and modify.
- **Readability and Documentation**: The use of clear variable names, f-strings, type hints, and descriptive docstrings makes the code self-documenting and a pleasure to read.

Suggestions for Enhancement

The script is already very strong, but here are a couple of minor suggestions that could enhance it further, following industry best practices for even more demanding scenarios.

1. **Implement Exponential Backoff for Retries**: The current retry mechanism uses a fixed delay. A more advanced strategy is "exponential backoff," where the delay increases after each failure (e.g., 2s, then 4s, then 8s). This can be more effective when an API is under heavy load, as it gives the server progressively more time to recover.

2. **Centralize User Communication**: The `fetch_exchange_rate` function currently prints status updates directly to the console. In larger applications, it's often better to have such functions return data or raise specific exceptions, and let the caller (the `if __name__ == "__main__"` block) handle all user-facing communication. This further improves modularity.

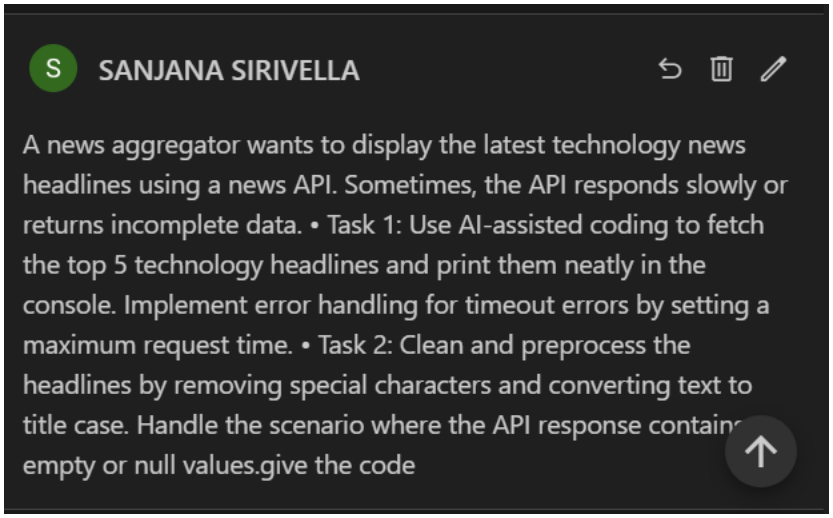
LAB-03

A news aggregator wants to display the latest technology news headlines using a news API. Sometimes, the API responds slowly or returns incomplete data.

• Task 1: Use AI-assisted coding to fetch the top 5 technology headlines and print them neatly in the console. Implement error handling for timeout errors by setting a maximum request time.

• Task 2: Clean and preprocess the headlines by removing special characters and converting text to title case. Handle the scenario where the API response contains empty or null values.

PROMPT:



CODE:

news_aggregator.py > ...

```python
import os
import re
import requests
from typing import List, Dict, Optional, Any

# --- Configuration ---
# IMPORTANT: You need a free API key from https://newsapi.org/
# Set it as an environment variable named NEWS_API_KEY
API_KEY = os.environ.get("NEWS_API_KEY", "YOUR_API_KEY")
API_URL = "https://newsapi.org/v2/top-headlines"
REQUEST_TIMEOUT_SECONDS = 5  # Max time to wait for a response

def fetch_tech_headlines(api_key: str) -> Optional[List[Dict[str, Any]]]:
    """
    Fetches the top 5 technology headlines from the NewsAPI.

    Args:
        api_key (str): Your API key for NewsAPI.

    Returns:
        Optional[List[Dict[str, Any]]]: A list of article dictionaries if successful,
                                        otherwise None.
    """
    if not api_key or api_key == "YOUR_API_KEY":
        print("Error: NewsAPI key is not configured. Please get a key from newsapi.org and set it.")
        return None

    params = {
        "apiKey": api_key,
        "category": "technology",
        "country": "us",
        "pageSize": 5
    }

    try:
        print("Fetching latest technology headlines...")
        response = requests.get(API_URL, params=params, timeout=REQUEST_TIMEOUT_SECONDS)

        # Raise an exception for HTTP errors (e.g., 4xx, 5xx)
        response.raise_for_status()

        data = response.json()

        if data.get("status") != "ok":
            print(f"API returned an error: {data.get('message')}")
            return None

        return data.get("articles")

    except requests.exceptions.Timeout:
        print(f"Error: The request timed out after {REQUEST_TIMEOUT_SECONDS} seconds. The API might be slow.")
        return None
    except requests.exceptions.RequestException as e:
        print(f"An error occurred during the API request: {e}")
        return None
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        return None
```

Select Encoding

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) Go Live

```python
def clean_and_format_headline(headline: Optional[str]) -> Optional[str]:
    """
    Cleans a headline by removing special characters and converting to title case.

    Args:
        headline (Optional[str]): The raw headline text.

    Returns:
        Optional[str]: The cleaned and formatted headline, or None if input is invalid.
    """
    # Task 2: Handle empty or null values
    if not headline:
        return None

    # Remove special characters, keeping letters, numbers, and spaces
    cleaned_text = re.sub(r'[^a-zA-Z0-9\s]', '', headline)

    # Convert to title case and remove extra whitespace
    return cleaned_text.strip().title()


# --- Main Execution ---
if __name__ == "__main__":
    # Task 1: Fetch headlines with error handling
    articles = fetch_tech_headlines(API_KEY)

    if articles:
        print("\n--- Top 5 Technology Headlines ---")
        headline_count = 0
```

```python
# --- Main Execution ---
if __name__ == "__main__":
    # Task 1: Fetch headlines with error handling
    articles = fetch_tech_headlines(API_KEY)

    if articles:
        print("\n--- Top 5 Technology Headlines ---")
        headline_count = 0
        for article in articles:
            # The API can sometimes return articles with null titles
            raw_title = article.get("title")

            # Task 2: Clean and preprocess the headline
            cleaned_title = clean_and_format_headline(raw_title)

            if cleaned_title:
                headline_count += 1
                print(f"{headline_count}. {cleaned_title}")

        if headline_count == 0:
            print("No valid headlines were found in the API response.")
    else:
        print("\nCould not display headlines due to an error.")
```

OUTPUT:

OBSERVATIONS:

General Observations

- **Code Quality & Readability**: The code is excellent. It's organized into logical functions (`fetch_tech_headlines`, `clean_and_format_headline`), uses descriptive names, and includes clear docstrings and type hints. This makes it very easy to understand and maintain.
- **Error Handling**: The error handling is comprehensive and a key strength of the script. You've correctly implemented a request timeout, used `raise_for_for_status()` for general HTTP errors, and handled specific API-level errors by checking the response payload.
- **Data Cleaning**: The `clean_and_format_headline` function is a great example of defensive programming. It correctly handles `None` or empty inputs and effectively sanitizes the text before display.
- **Configuration Management**: Using `os.environ.get` to manage the API key is a security best practice. It cleanly separates configuration from the code itself.

Suggestions for Improvement

The script is already very effective, but we can make a few small adjustments to align it even more closely with production-level coding standards

1. **Use the `logging` Module**: For applications that might be run as part of an automated process, the `logging` module is more powerful than `print()`. It allows you to categorize messages (e.g., `INFO` for status, `ERROR` for failures) and easily redirect output to a file for later inspection
2. **Exit Early on Critical Configuration Errors**: If the API key isn't set, the script can't do anything useful. It's better to exit immediately with an error message rather than letting the program continue.
3. **Refine the Cleaning Regex**: The NewsAPI sometimes appends a character count like `[+1234 chars]` to truncated headlines. We can refine the regular expression in the cleaning function to remove this specific pattern for an even cleaner output.

----------THE END----------